# cat

- The cat command can be used to create a file, write data into it, and display the contents of a regular file.
    - cat file_name: Reads file content if it is present.
    - cat > file_name: Creates a file and writes into it.
    - cat >> file_name: Appends data into the file if it is present.

# tac

- print Contents of Files in reverse

```
Command :$tac file_name
Output : prints the contents of the file in reverse order.
```

# nl

- Number the lines of a file

```
Command :$nl file_name
Output : prints the contents of the file with line numbers.
```

# Redirection Operators:

- OS by default opens 3 files for any program:

    1. stdin ( 0 ): Standard input buffer
        - by default, any program accepts/reads input from the stdin file/buffer.
    2. stdout( 1 ): Standard output buffer
        - by default, any program writes data into the stdout buffer.
    3. stderr( 2 ): Standard error buffer
       - if any program encounters errors, those errors are written by default into the "stderr" buffer, and the contents of the stderr buffer are displayed on the standard output.

- Three Redirection Operators are available:

    1. Input redirection operator (<) - Left chevron.
    2. Output redirection operator (>) - Right chevron.
    3. Error redirection operator (2>) - Redirects error messages to a file or another output stream.

- When any process terminates, its exit status is returned to the OS.

    - A program can return any one of the following three values upon exit:
      - Exit value 0 : Successful termination.
      - Exit value > 0 : Unsuccessful termination.
      - Exit value < 0 : Abnormal termination.

- The exit value of the last executed command is stored in a special variable called "$?".
- The exit value of the last executed command can be checked by using the echo command as shown below:

```
Command :$echo $?
Output : prints the exit value of the last executed command.
```

- Examples of Redirection Operators:
    - Example of Input Redirection:
      - Command :$cat < file_name
      - Output : prints the contents of the file.
    - Example of Output Redirection:
      - Command :$ls > file_name
      - Output : creates a file with the name "file_name" and writes the output of the "ls" command into it.
    - Example of Error Redirection:
      - Command :$ls /root 2> error_file
      - Output : creates a file with the name "error_file" and writes the error message of the "ls /root" command into it.

# pipe

- The pipe operator (|) is used to redirect the output of one command to the input of another command.
- The pipe command (|) creates a temporary buffer for storing the output of one command, which can then be passed as input to another command.
- Using a pipe, we can redirect the output of any command to the input of another command.
- Two processes are connected using the pipe operator (|).
- Both processes run simultaneously and are automatically rescheduled as data flows between them.
- Without pipes, you would need several steps to perform a single task.
    - Example:

```
$who | wc
    - Counts the number of users currently logged in.
$echo "hello" | wc -c
    - Counts the number of characters in the string "hello".
$ls -l | wc
    - Counts the number of lines, words, and characters in the output
of ls -l.
```

- Multiple pipes can be used in a single command.

# Cascadding of following commands with pipe command:

- head
    - The head command is used to view the first few lines of a file. By default, it shows the first 10 lines.
        - e.g.head [options] [file]
            - head file.txt
            - head -n 5 file.txt
            - cat file.txt | head -n 20
            - ls -l | head -n 5
            - ps -ef | head -n 10

- tail

- tail command shows the last few lines of a file. By default, it displays the last 10 lines.
- Usefull in Monitoring logs in real-time ,Checking recent data entries
- e.g.tail [options] [file]
- tail file.txt
- tail -n 5 file.txt
- cat file.txt | tail -n 5
- ls -l | tail -n 5
- ps -ef | tail -n 10

    - tail -f file.txt
      - Continuously monitor a file for new data entries.

- sort
    - The sort command arranges lines in alphabetical or numerical order.
        - e.g sort [options] [file]
            - sort file.txt
                - Sorts lines in ascending alphabetical order.
            - sort -n marks.txt
                - Sort numerically
            - sort -k 2 names.txt
                - Sort by second field/column in the file.
            - cat names.txt | sort -k 2
                - Sort by second field/column using pipe.
            - sort -r file.txt
                - Sorts lines in descending order.

- uniq
    - The uniq command filters out repeated lines in a file ** but only if they are consecutive**.
        - e.g uniq [options] [file]

        - sort file.txt | uniq
            - Remove duplicate lines from a sorted file.
        - sort names.txt | uniq -c
            - Count how many times each name appear
        - sort data.txt | uniq -d
            - Show only lines that occur more than once.
        - sort data.txt | uniq -u
            - shows only the lines that appear exactly once

- less
    - View Large Text Files Page by Page
    - Use less when dealing with long outputs from logs, help docs, or system status commands.
        - options:
            - use UP/Down arrow =>Scroll line up/down
            - space => Next page
            - b => Previous page
            - /word => Search for "word"
            - n => Next search result
            - q => Quit viewer
        - e.g
            - less file.txt
            - e.g less [file]
              - Opens the file in a scrollable viewer.
            - cat bigfile.txt | less
              - Scroll through output of any command.
            - man ls | less
              - View long manual pages.

- cut
    - Extract Specific Fields or Characters
    - The cut command is used to extract columns, fields, or character ranges from each line of a file or input.
        - e.g.cut [options] [file]
            - echo "DBDA DESD DAC DMC DITISS" | cut -d " " -f1
                - Extracts fields 1 using comma (" ") as a delimiter.
            - echo "DBDA DESD DAC DMC DITISS" | cut -d " " -f2
            - echo "DBDA DESD DAC DMC DITISS" | cut -d " " -f5
            - echo "DBDA DESD DAC DMC DITISS" | cut -d " " -f1,2
                - Extracts fields 1 and 2, using comma (" ")as a delimiter.
            - echo "DBDA DESD DAC DMC DITISS" | cut -d " " -f2,3
            - cut -d ',' -f1,3 file.csv
                - Extracts fields 1 and 3, using comma (,) as a delimiter.
            - ls -l -i -s | cut -d " " -f2
                - Extracts the second field (file size) from the output of ls -l -i -s command.

- tr

    - The tr (translate) command is used to convert, squeeze, or delete characters from input.
    - tr works only with stdin (standard input). You must use input redirection or pipes.
        - e.g tr [options] SET1 [SET2]
            - echo "linux" | tr 'a-z' 'A-Z' : Convert sting uppercase
            - echo "HELLO" | tr 'A-Z' 'a-z' : Convert string lowercase
            - echo "hi there" | tr -s ' ' : Squeeze Repeated Characters (-s)
            - echo "hello123" | tr -d '0-9' : Remove Characters
            - tr 'a-z' 'A-Z' < file.txt : Converts all text in a file to uppercase

# wc

- The wc (word count) command is used to count lines, words, and characters in a file or input.
    - e.g wc [options] [file]
    - wc file.txt
        - Displays the number of lines, words, and characters in the file.
    - wc -l file.txt
        - Displays only the number of lines in the file.
    - wc -w file.txt
        - Displays only the number of words in the file.
    - wc -c file.txt
        - Displays only the number of characters in the file.
    - echo "Hello World" | wc
        - Counts lines, words, and characters in the string "Hello World".
    - echo "Hello World" | wc -w
        - Counts only the number of words in the string "Hello World".

# alias

- Create Command Shortcut
- The alias command is used to create shortcuts for frequently used commands or command combinations.
    - Syntax: alias name='command'
    - alias ls_shrt='ls -l -a -h -s'
        - Creates a shortcut for a detailed ls command.
    - alias hi='echo "Hey $USER, welcome back!"
        - Creates a shortcut for a greeting message.

# unalias

- Remove Command Shortcut
- The unalias command is used to remove previously created command shortcuts (aliases).
    - Syntax: unalias name
    - unalias ls_shrt
        - Removes the alias named ls_shrt.

        - unalias hi
            - Removes the alias named hi.

# file

- file command is used to determine the type of a file
- whether it's a text file, executable, image, archive, etc.
    - e.g file [filename]
    - file notes.txt
    - notes.txt
    - file script.sh